

EXECUTION PLAN

WebGenie

AI Website Builder — Full-Stack Build Guide

React + Vite + FastAPI + MongoDB Atlas + OpenRouter

Prepared for Anish · September 2026

Repo: [WebGenie](#) · Local build, deploy later

Table of Contents

0	How This Plan Works (Manual → Code → Test)
1	What WebGenie Is
2	Tech Stack & All Packages
3	Complete Folder Structure (Client + Server) & Component/Page Folder Convention
4	Data Model — MongoDB Documents
5	REST API Contract
6	WebSocket Message Contracts
7	Design System
8	Verified Technical Notes & Gotchas
9	Phase 0 → Phase 12: Step-by-Step Build Plan
10	Final End-to-End Test Checklist
11	Resume Bullet (once built)

0. How This Plan Works

Every phase below is a **self-contained, visible deliverable** — you never move to the next phase until the current one is coded and you've verified it works, including how it looks. Each phase is split into three parts:

Label	Meaning
MANUAL — YOU	Something only you can do: creating accounts, clicking through a cloud console, copying an API key. Claude Code cannot do these for you.
CODE — US	Everything else: scaffolding, installing packages, writing files, running dev servers. Done together in the coding session — Claude Code runs the terminal commands and writes the files directly.
TEST — YOU	You open the browser (and sometimes the database) and confirm the phase's checkpoint actually works, including how the UI looks and feels, before we move on.

Rule of thumb: if a step needs a browser tab open to a third-party website (MongoDB Atlas, OpenRouter) or a decision only you can make, it's **Manual**. If it's a terminal command or a file inside this repo, it's **Code** and happens in the session.

1. What WebGenie Is

A SaaS web app where a user types a prompt like *"create a developer portfolio website"* and the AI:

1. **Plans** a file structure (which components the site needs)
2. **Generates each file's code in parallel**, streaming progress to the UI in real time
3. **Renders the result live** using Sandpack (an in-browser React sandbox, not a static preview)
4. Lets the user **manually edit code** in Sandpack's built-in editor, or
5. Continue **chatting with the AI** to revise the site (AI returns precise create/update/delete file operations, not a full rewrite)
6. **Export** the project as a downloadable Vite+React zip, or **Publish** it read-only

Learning path this plan follows: Basic React Project → Pages/Router → Login/Register UI → Home Page UI → Builder Page → Preview/Publish Page → Backend Server → Auth APIs → AI Build API → OpenRouter Integration → Agent Chat API → Full Integration.

2. Tech Stack & All Packages

Layer	Choice	Notes
Frontend	React 18 + TypeScript + Vite	React pinned to 18, not Vite's default 19 (Sandpack compatibility — see §8); <code>react-ts</code> Vite template
Styling	Tailwind CSS v4	CSS-first config, no <code>tailwind.config.js</code>
Routing	React Router v6	Installed explicitly as <code>@6</code> (plain install grabs v7); ships its own types
Global state	Redux Toolkit + react-redux	Typed store/slices for auth session + active project — see §3, §8
HTTP client	Axios	<code>withCredentials: true</code> , cookie-based sessions
Live preview + editor	Sandpack (<code>@codesandbox/sandpack-react</code>)	Frontend-only in-browser bundler/sandbox
Export	JSZip + file-saver	100% client-side, builds a real Vite project zip
Backend framework	FastAPI (Python)	Async, auto-docs at <code>/docs</code>
Database	MongoDB Atlas (free M0 cluster)	Async driver: Motor
Auth	JWT in httpOnly cookie	<code>passlib[bcrypt]</code> + <code>python-jose</code>
Real-time updates	WebSocket (native FastAPI)	<code>plan</code> → <code>file_start</code> → <code>file_complete</code> → <code>done</code>
AI model	OpenRouter free model	One specific pinned model id, not the <code>:free</code> router
Structured AI output	<code>instructor</code> + Pydantic	Patches an OpenAI-compatible client to validate model output into a schema

Frontend packages (installed across Phases 1–5)

```
react@18
react-dom@18
tailwindcss
@tailwindcss/vite
react-router-dom@6
@reduxjs/toolkit
react-redux
clsx
tailwind-merge
axios
@codesandbox/sandpack-react
jszip
file-saver

# dev dependencies (types)
@types/react@18
@types/react-dom@18
@types/file-saver
```

Backend packages — `server/requirements.txt` (Phase 7)

```
fastapi[standard]
motor
pydantic
python-dotenv
passlib[bcrypt]
bcrypt==4.0.1
python-jose[cryptography]==3.3.0
instructor
openai
```

Pins on `bcrypt` and `python-jose` avoid known crashes/security gaps — see §8.

3. Complete Folder Structure

This project uses **TypeScript** (not plain JS) and **Redux Toolkit** (not Context) for global state — the "best practice" stack. The folder categories follow the organizing principle you asked for (one clear home per kind of code): **pages/** are full screens, **components/** are their reusable building blocks, **services/** is the only place that talks to the backend, **store/** holds Redux state shared across screens, **constants/** centralizes routes/endpoints/keys, **hooks/** holds reusable React logic, **types/** holds shared TypeScript types, and **utils/** holds plain framework-free helpers. Every entry under **components/** and **pages/** is itself a folder, not a flat file — see §3.1 right after the tree.

Client — `client/src/`

```
src/
├─ main.tsx          # App entry – boots React, Router, and the Redux <Provider>
├─ App.tsx           # Top-level layout: Navbar + route outlet
├─ assets/           # Static images/icons (e.g. logo.svg)
├─ components/       # Reusable UI pieces – one FOLDER per component (§3.1)
│ └─ Navbar/
│ └─ ProtectedRoute/ # Route guard – logic only, no .styles.ts
│ └─ ProjectCard/
│ └─ ChatPanel/      # Left panel of Builder page
│ └─ BuilderToolbar/  # Save / Preview / Download / Publish buttons
│ └─ DeviceToggle/
├─ constants/        # Fixed values, defined once
│ └─ routes.ts        # ROUTES.HOME, ROUTES.LOGIN, ROUTES.BUILDER(id), ...
│ └─ apiEndpoints.ts  # API_ENDPOINTS.AUTH.LOGIN, ...
│ └─ storageKeys.ts   # localStorage key names
├─ hooks/            # Reusable React hooks
│ └─ useAuth.ts       # Thin hook wrapping the auth slice
│ └─ useWebSocket.ts  # Generic WS connect/subscribe/cleanup hook
│ └─ useDebounce.ts   # Debounced value, used for autosave in Builder
├─ pages/            # One FOLDER per route, same pattern as components/ (§3.1)
│ └─ HomePage/
│ └─ LoginPage/
│ └─ RegisterPage/
│ └─ MyProjectsPage/
│ └─ CommunityPage/
│ └─ BuilderPage/     # Sandpack lives here
│ └─ PublicPreviewPage/
│ └─ NotFoundPage/
├─ routes/           # The single route table
│ └─ AppRoutes.tsx
├─ services/         # The API layer – ALL backend calls live here
│ └─ api.ts           # axios instance (baseUrl '/api', withCredentials)
│ └─ authService.ts   # register/login/logout/me
│ └─ projectService.ts # list/create/get/publish/delete/saveFiles
│ └─ socketService.ts  # builds ws:// URLs for generate/chat sockets
├─ store/            # Global state shared across screens (Redux Toolkit)
│ └─ index.ts         # configureStore + RootState/AppDispatch types
│ └─ hooks.ts         # typed useAppDispatch / useAppSelector
│ └─ slices/
│   └─ authSlice.ts   # { user, status } + login/register/logout thunks
│   └─ projectSlice.ts # active project + files/messages/version state
├─ styles/
│ └─ index.css        # Tailwind import + GLOBAL/SHARED design system (§7) –
│                       # the one place common classes (.btn-primary, .card, ...) live
├─ types/            # Shared TypeScript types
│ └─ auth.types.ts    # User, AuthState
│ └─ project.types.ts  # Project, FileEntry, Message (mirrors Mongo schema)
│ └─ api.types.ts     # shared request/response shapes
├─ utils/            # Pure helper functions
│ └─ cn.ts            # shared class-merge helper (clsx + tailwind-merge), see §3.1
│ └─ sandpackUtils.ts # maps {content,hash} → Sandpack's {code} shape
│ └─ exportProject.ts  # builds & downloads the Vite+React zip
│ └─ validators.ts     # email/password/prompt validation helpers
```

3.1 — Component & Page Folder Convention

Every component and page is its own folder of exactly three files (a fourth only if genuinely needed — e.g. a page-local constants file):

```
ComponentName/
├─ ComponentName.tsx      # component logic/JSX
├─ ComponentName.styles.ts # named string constants – the ONLY place literal Tailwind classes live
└─ index.ts              # export { default } from "./ComponentName";
```

```
// ComponentName.styles.ts
export const styles = {
```

```

    container: "flex items-center justify-between p-4 rounded-lg border border-border",
    title: "text-lg font-semibold text-ink-900",
  } as const;

// ComponentName.tsx
import { styles } from "../ComponentName.styles";
<div className={styles.container}>
  <h2 className={styles.title}>{title}</h2>
</div>

```

Why this is a hard rule, not a style preference: Tailwind v4's scanner statically greps source files for complete class-name strings to decide what CSS to generate — it cannot resolve runtime interpolation. `'text-${color}-500'` silently produces **no CSS** for that class. So every string in a `.styles.ts` file must be a complete, literal class list. For a class that varies by prop/state, don't interpolate a fragment — pick between fully-written literal strings via a lookup map or function:

```

const STATUS_COLOR: Record<Status, string> = {
  generating: "text-primary-600",
  completed: "text-ink-700",
  failed: "text-red-600",
};
export const styles = {
  badge: (status: Status) => `${STATUS_COLOR[status]} text-xs font-semibold`,
} as const;

```

The function's return value is computed at runtime, but every fragment it's built from is a complete literal the scanner can see somewhere in the file — that's what the scanner actually needs, not a static final string.

When a caller-supplied `className` needs to merge with a component's own classes, use the shared `cn()` helper (`clsx` + `tailwind-merge`, in `utils/cn.ts`) instead of string concatenation: `className={cn(styles.container, className)}`.

Shared/common classes stay in `styles/index.css` (\$7's `.btn-primary` / `.btn-secondary` / `.card` / `.input-field`) — never duplicated per component. A component's own `.styles.ts` can reference them as literal strings alongside its component-specific classes, e.g. `submitButton: "btn-primary w-full"`.

A component with no meaningful styling of its own (e.g. `ProtectedRoute`, a pure logic wrapper) skips `.styles.ts` entirely — don't create an empty file. If two pages end up duplicating the same non-trivial class strings (e.g. Login/Register share form styling), factor the shared ones into `styles/shared.styles.ts` and import from both — add this only once duplication actually shows up, not preemptively.

Server — [server/app/](#)

```

server/
├─ app/
│  ├─ main.py           # FastAPI app, CORS, router mounting
│  ├─ config.py         # loads .env (Mongo URI, OpenRouter key, JWT secret)
│  ├─ database.py       # Motor client + collection getters
│  ├─ models.py         # Pydantic document models (User, Project, FileEntry, Message)
│  ├─ auth/
│  │  ├─ security.py    # password hashing, JWT encode/decode
│  │  └─ dependencies.py # get_current_user() from cookie
│  ├─ routers/
│  │  ├─ auth_router.py
│  │  ├─ projects_router.py
│  │  ├─ community_router.py
│  │  └─ ws_router.py   # the two WebSocket endpoints
│  └─ ai/
│     ├─ schemas.py     # Pydantic: FilePlanSchema, FileCodeSchema, RevisionResultSchema, FileOpSchema
│     ├─ prompts.py     # system prompts (design-quality rules)
│     ├─ service.py     # generate_project(), revise_project()
│     ├─ normalizer.py  # fixes double-escaped newlines/quotes
│     ├─ validator.py   # auto-fixes class=, void tags, default export
│     └─ diff.py        # apply_operations(), md5 hashing
├─ requirements.txt
└─ .env                # MONGODB_URI, JWT_SECRET, OPENROUTER_API_KEY, ...

```

4. Data Model — MongoDB Documents

Collection: **users**

```
{
  "_id": "ObjectId",
  "name": "string",
  "email": "string (unique, indexed)",
  "hashed_password": "string (bcrypt)",
  "created_at": "datetime"
}
```

Collection: **projects**

```
{
  "_id": "ObjectId",
  "owner_id": "ObjectId (ref: users._id)",
  "name": "string",
  "description": "string (the original prompt)",
  "version": "int (incremented on every AI or manual edit)",
  "status": "generating | completed | failed",
  "published": "bool",
  "files": {
    "/App.js": { "content": "string", "hash": "string (md5, first 12 chars)" },
    "/styles.css": { "content": "string", "hash": "string" }
  },
  "messages": [
    { "role": "user", "content": "string", "timestamp": "datetime" },
    { "role": "assistant", "content": "string", "timestamp": "datetime" }
  ],
  "created_at": "datetime",
  "updated_at": "datetime"
}
```

Files and messages are embedded directly in the project document (not separate collections) so one query loads everything the Builder page needs.

5. REST API Contract

Method	Path	Auth	Purpose
POST	/api/auth/register	No	{name,email,password} → creates user, sets cookie, returns {user}
POST	/api/auth/login	No	{email,password} → verifies, sets cookie, returns {user}
POST	/api/auth/logout	Yes	Clears session cookie
GET	/api/auth/me	Yes	Returns current user (session restore on refresh)
GET	/api/projects	Yes	List current user's projects (summary fields only)
POST	/api/projects	Yes	{prompt} → creates shell in generating, returns {project_id}
WS	/ws/projects/{id}/generate	Yes	Streams plan/file_start/file_complete/done events
GET	/api/projects/{id}	Yes (owner)	Full project incl. files + messages
WS	/ws/projects/{id}/chat	Yes (owner)	Agent Chat API — revision prompt in, file ops out
PUT	/api/projects/{id}/files	Yes (owner)	Manual code-editor save
PATCH	/api/projects/{id}/publish	Yes (owner)	Toggle published
GET	/api/projects/public/{id}	No	Read-only published view
DELETE	/api/projects/{id}	Yes (owner)	Delete project

Export has **no backend endpoint** — handled entirely client-side by `utils/exportProject.js`.

6. WebSocket Message Contracts

/ws/projects/{id}/generate

```
// server → client, in order
{ "type": "plan", "files": ["/App.js", "/styles.css", "/components/Header.js"] }
{ "type": "file_start", "path": "/App.js" }
{ "type": "file_complete", "path": "/App.js", "code": "... " }
{ "type": "file_start", "path": "/styles.css" }
{ "type": "file_complete", "path": "/styles.css", "code": "... " }
{ "type": "done", "project": { "...": "full project document" } }
```

/ws/projects/{id}/chat

```
// client → server
{ "prompt": "make the header dark blue" }

// server → client
{
  "type": "revision_done",
  "operations": [
    { "op": "update", "path": "/components/Header.js", "search": "... ", "replace": "... " }
  ],
  "project": { "...": "updated project document, version bumped" }
}
```

7. Design System

Applied from Phase 2 onward in [src/styles/index.css](#). CSS-variable-driven Tailwind v4 theme — no component library, no [tailwind.config.js](#).

```
@import url('https://fonts.googleapis.com/css2?family=Urbanist:wght@400;500;600;700;800&display=swap');
@import "tailwindcss";

@theme {
  --font-sans: "Urbanist", ui-sans-serif, system-ui, sans-serif;
  --color-primary-500: #6366f1;
  --color-primary-600: #4f46e5;
  --color-primary-700: #4338ca;
  --color-ink-900: #0f172a;
  --color-ink-700: #334155;
  --color-ink-400: #94a3b8;
  --color-surface-0: #ffffff;
  --color-surface-50: #f8fafc;
  --color-surface-100: #f1f5f9;
  --color-border: #e2e8f0;
  --radius-card: 0.75rem;
  --shadow-card: 0 1px 2px 0 rgb(0 0 0 / 0.04), 0 1px 3px 0 rgb(0 0 0 / 0.06);
}

body { @apply bg-surface-50 text-ink-900 font-sans antialiased; }
.btn-primary { @apply inline-flex items-center justify-center rounded-lg bg-primary-600 px-4 py-2 text-sm font-semibold text-white shadow-sm
hover:bg-primary-700 transition-colors disabled:opacity-50 disabled:pointer-events-none; }
.btn-secondary { @apply inline-flex items-center justify-center rounded-lg border border-border bg-surface-0 px-4 py-2 text-sm font-semibold
text-ink-700 hover:bg-surface-100 transition-colors; }
.card { @apply rounded-card border border-border bg-surface-0 p-6 shadow-card; }
.input-field { @apply w-full rounded-lg border border-border bg-surface-0 px-3 py-2 text-sm text-ink-900 placeholder:text-ink-400
focus:outline-none focus:ring-2 focus:ring-primary-500; }
```

Every page/component reuses `.btn-primary` / `.btn-secondary` / `.card` / `.input-field` instead of ad-hoc Tailwind strings, so the whole app reads as one consistent system.

8. Verified Technical Notes & Gotchas

These were checked against current (Sept 2026) package behavior rather than assumed — keep them in mind as the relevant phase comes up.

React 18, not 19. `npm create vite` scaffolds React 19 by default, but `@codesandbox/sandpack-react`'s peer deps top out at React 18 (the project is effectively unmaintained). Immediately after scaffolding in Phase 1: `npm install react@18 react-dom@18`.

Tailwind v4 has no config file. No `tailwind.config.js`, no PostCSS setup, no `npx tailwindcss init`. Just `@tailwindcss/vite` as a Vite plugin + `@theme {}` in CSS.

Sandpack's `files` prop expects `{code}`, not `{content}`. Our Mongo schema stores `content`; `sandpackUtils.js` must map between the two. Also put `key={project.version}` on `SandpackProvider` so every edit forces a clean remount (files aren't reliably reactive otherwise). Use `template="react"` (classic bundler) for the live preview — faster than `vite-react` for many small hot-swapped files.

Cross-origin cookies are fragile on localhost. Frontend (:5173) and backend (:8000) are different origins. Instead of fighting `SameSite=None; Secure` on plain HTTP, proxy both REST and WS through Vite's dev server so the browser only ever talks to :5173:

```
// client/vite.config.js
server: {
  proxy: {
    '/api': { target: 'http://localhost:8000', changeOrigin: true },
    '/ws': { target: 'ws://localhost:8000', ws: true },
  },
}
```

Then the cookie can simply be `httponly=True, samesite="lax", secure=False` for local dev.

instructor + OpenRouter: use the current, non-deprecated pattern:

```
client = instructor.from_provider(
    f"openrouter/{settings.OPENROUTER_MODEL}",
    base_url="https://openrouter.ai/api/v1",
    api_key=settings.OPENROUTER_API_KEY,
    async_client=True,
)
result = await client.chat.completions.create(
    response_model=FilePlanSchema,
    messages=[...],
    extra_body={"provider": {"require_parameters": True}},
)
```

The `extra_body` line forces the routed free model to actually honor structured-output params — without it some free models silently ignore the schema.

Pin one exact free model id — never `openrouter/free`. That meta-router picks randomly among free models each call, which breaks a schema-constrained pipeline. Browse `openrouter.ai/models`, filter Free + Tool-calling, copy one id into `.env` as `OPENROUTER_MODEL` (Phase 0).

requirements.txt pins that prevent crashes: `bcrypt==4.0.1` (newer bcrypt removed an attribute `passlib` reads, crashing on first hash/verify) and `python-jose[cryptography]==3.3.0` (use the real crypto backend, not the pure-Python fallback).

Windows PowerShell `venv` activation. `.\venv\Scripts\Activate.ps1` fails on a fresh machine with an execution-policy error. Run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` immediately before activating (process-scoped, no admin rights needed).

Install React Router explicitly as v6. Plain `npm install react-router-dom` grabs v7 by default; run `npm install react-router-dom@6`. It ships its own TypeScript types.

TypeScript scaffold + type packages. Use `npm create vite@latest client -- --template react-ts` (not `react`). After pinning React to 18, also install matching types: `npm install -D @types/react@18 @types/react-dom@18`. Sandpack and Axios ship their own types; `file-saver` does not, so add `npm install -D @types/file-saver`.

Redux Toolkit pattern. `store/index.ts` builds the store with `configureStore` and exports `RootState` / `AppDispatch` inferred from it. `store/hooks.ts` exports typed `useAppDispatch` / `useAppSelector` — components use these, never the raw untyped `useDispatch` / `useSelector`. Each domain gets its own slice (`authSlice.ts`, `projectSlice.ts`) built with `createSlice` + `createAsyncThunk` for API calls, so `services/` stays plain request functions and the slice owns loading/error state.

Every component/page is a folder, not a flat file — see §3.1 for the full convention. In short: `Name/Name.tsx` + `Name.styles.ts` (named Tailwind class-string constants) + `index.ts` (re-export). Requires `clsx` + `tailwind-merge` for the shared `cn()` class-merge helper in `utils/cn.ts`.

9. Phase 0 → Phase 12: Step-by-Step Build Plan

Confirmed local toolchain: Node v22.13.1, npm 11.6.4, Python 3.13.0 — all sufficient, no installs needed for those.

CHUNK 0 — Setup (no code, no UI yet)

Phase 0 — Accounts & Environment

Deliverable: five credential values ready in hand. No UI checkpoint — this phase is pure setup.

MANUAL — YOU

- Create a **MongoDB Atlas** account → free **M0** cluster → create a DB user (username/password) → Network Access: allow your current IP (or `0.0.0.0/0` for local dev convenience) → copy the connection string (`mongodb+srv:// ...`) → decide a database name, e.g. `webgenie` .
- Create an **OpenRouter** account at openrouter.ai → generate an API key.
- Browse `openrouter.ai/models` , filter **Free** + **Tool calling** , pick one specific current model id (not `openrouter/free`) and write it down.
- Generate a random 32+ character JWT secret string.
- Confirm you have the five values ready: `MONGODB_URI` , `MONGODB_DB_NAME` , `JWT_SECRET` , `OPENROUTER_API_KEY` , `OPENROUTER_MODEL` .

CODE

None — nothing to build yet.

TEST — YOU

Log into the Atlas dashboard and confirm the M0 cluster shows "Active"; confirm the OpenRouter dashboard shows your new key.

CHUNK 1 — Frontend Shell (Phases 1–3)

Phase 1 — Vite + React + TypeScript Scaffold + Design System

Deliverable: a styled placeholder page proving the toolchain works.

MANUAL: NONE

CODE — US

Files: `client/vite.config.ts` , `tsconfig.json` (generated), `src/styles/index.css` , `src/utils/cn.ts` , `src/main.tsx` , `src/App.tsx` .

```
npm create vite@latest client -- --template react-ts
cd client
npm install
npm install react@18 react-dom@18
npm install -D @types/react@18 @types/react-dom@18
npm install tailwindcss @tailwindcss/vite
npm install clsx tailwind-merge
```

Add `tailwindcss()` to the Vite plugins array; add the `/api` proxy entry now (§8) even though nothing calls it yet. `index.css` gets the full design system from §7. `utils/cn.ts` defines the shared `cn()` helper (§3.1) every component will use from Phase 2 onward. `App.tsx` renders a "WebGenie" placeholder heading using `.card` / `.btn-primary` to prove styling is wired.

TEST — YOU

`npm run dev` → open `http://localhost:5173` → confirm the placeholder renders in the Urbanist font with the indigo primary color and card shadow (not default unstyled HTML), and that there are no TypeScript compile errors in the terminal.

Phase 2 — Pages, Routing, Navbar

Deliverable: a clickable app skeleton — every page reachable, empty but styled.

MANUAL: NONE

CODE — US

Files (every page/component follows the §3.1 folder pattern):

`src/pages/{HomePage, LoginPage, RegisterPage, MyProjectsPage, CommunityPage, BuilderPage, PublicPreviewPage, NotFoundPage}/{Name}.tsx, {Name}.styles.ts` (placeholder shells), `src/components/Navbar/{Navbar}.tsx, {Navbar}.styles.ts, index.ts`, `src/routes/AppRoutes.tsx`, `src/constants/routes.ts`, `src/App.tsx` (wires router).

```
npm install react-router-dom@6
```

`constants/routes.ts` centralizes every path as typed constants/functions (`ROUTES.HOME`, `ROUTES.BUILDER(id: string)`, ...) so no page ever hardcodes a URL string. `AppRoutes.tsx` is the single route table consumed by `App.tsx`, wrapped in `BrowserRouter` with `Navbar` always visible. Each folder's `index.ts` is `export { default } from './HomePage';` — consumers import the folder (`import HomePage from '@pages/HomePage'`), never the inner file directly.

TEST — YOU

Click every Navbar link and also type each URL directly in the address bar — every route renders its placeholder heading, no console errors, no broken links.

Phase 3 — Redux Store + Login / Register UI (mock mode)

Deliverable: working-feeling auth forms, no backend yet.

MANUAL: NONE

CODE — US

Files: `src/store/{index, hooks}.ts`, `src/store/slices/authSlice.ts`, `src/types/auth.types.ts`, `src/services/api.ts`, `src/services/authService.ts`, `src/hooks/useAuth.ts`, `src/components/ProtectedRoute/{ProtectedRoute}.tsx, index.ts` (no `.styles.ts` — pure logic wrapper, §3.1), `src/pages/{LoginPage, RegisterPage}/{Name}.tsx, {Name}.styles.ts, index.ts` (fleshed out), `src/utils/validators.ts`.

```
npm install @reduxjs/toolkit react-redux
```

`services/api.ts`: `axios.create({ baseURL: '/api', withCredentials: true })`. `store/slices/authSlice.ts` defines `AuthState { user: User | null, status }` with a `createAsyncThunk` per auth action; in mock mode the thunks resolve a fake user locally — no network call yet, so the rest of the UI can be built against a stable shape before Phase 8 swaps in real API calls. `store/index.ts` wires `configureStore` and exports `RootState / AppDispatch`; `store/hooks.ts` exports the typed `useAppDispatch / useAppSelector` (§8). `hooks/useAuth.ts` wraps the slice into a simple `{user, login, register, logout, status}` API for components to consume. `ProtectedRoute.tsx` redirects to `/login` when `user` is null, guarding `/projects` and `/builder/:id`. `LoginPage.styles.ts / RegisterPage.styles.ts` hold each form's own literal classes, referencing `.input-field / .btn-primary` from the global stylesheet.

TEST — YOU

Submit both forms with valid input → redirected to `/projects`; Navbar shows logged-in state (name + logout button) driven by Redux; try visiting `/projects` logged-out first — confirm you're bounced to `/login`. All without a backend running.

CHUNK 2 — Frontend Core (Phases 4–6, the biggest FE work)

Phase 4 — Home Page UI

Deliverable: the marketing/landing hero with a working prompt box.

MANUAL: NONE

CODE — US

Files: `src/pages/HomePage/{HomePage}.tsx, {HomePage}.styles.ts, index.ts` (fleshed out), `src/components/ProjectCard/{ProjectCard}.tsx, {ProjectCard}.styles.ts, index.ts` (built now, reused in Phases 6 & 12), `src/types/project.types.ts` (early `Project / FileEntry` types mirroring the Mongo schema).

Hero section + large prompt textarea + "Generate" button, all using `.card / .input-field / .btn-primary`. In mock mode, submitting navigates to `/builder/mock-id` carrying the prompt in state/context.

TEST — YOU

Type a prompt, click Generate → lands on the (still placeholder) Builder page with the prompt visible somewhere (e.g. as the first chat message) — proves page-to-page state hand-off before Sandpack exists.

Phase 5 — Builder Page (Sandpack + Chat + Toolbar)

Deliverable: a live, editable code sandbox with working zip export — the heaviest frontend phase.

MANUAL: NONE

CODE — US

Files: `src/pages/BuilderPage/{BuilderPage.tsx, BuilderPage.styles.ts, index.ts}`,
`src/components/{ChatPanel, BuilderToolbar, DeviceToggle}/{Name.tsx, Name.styles.ts, index.ts}`,
`src/utils/{sandpackUtils, exportProject}.ts`, `src/store/slices/projectSlice.ts` (mock project state, managed via Redux now).

```
npm install @codesandbox/sandpack-react
npm install jszip file-saver
npm install -D @types/file-saver
```

`sandpackUtils.ts`: a typed `toSandpackFiles(files: Record<string, FileEntry>)` mapper (§8) + a light Sandpack theme matching the design tokens. `BuilderPage.tsx` holds mock `project` state (`/App.js` + `/styles.css`) via `projectSlice` and renders:

```
<SandpackProvider key={project.version} template="react" theme={webgenieSandpackTheme} files={toSandpackFiles(project.files)}>
  <SandpackLayout>
    <SandpackCodeEditor />
    <SandpackPreview />
  </SandpackLayout>
</SandpackProvider>
```

...plus `ChatPanel` (mock-echo replies for now) and `BuilderToolbar` (Export / Publish stub / `DeviceToggle`). `exportProject.ts` is fully real already (100% client-side, typed against the `Project` type): builds a real Vite+React project zip via JSZip (own `package.json`, `vite.config.js`, `index.html`, `src/main.jsx`, plus every project file) and downloads it via `file-saver`.

TEST — YOU

1. Mock code renders live in the preview pane.
2. Editing code in the editor live-updates the preview.
3. Click Export → unzip the downloaded file → run `npm install && npm run dev` inside it → confirm it's a real, independently working Vite+React app. This is the true correctness gate for export since there's no backend to catch mistakes.

Phase 6 — Public Preview + Publish Toggle UI

Deliverable: a read-only shareable preview page.

MANUAL: NONE

CODE — US

Files: `src/pages/PublicPreviewPage/{PublicPreviewPage.tsx, PublicPreviewPage.styles.ts, index.ts}` (read-only `SandpackPreview` only, no editor/chat), `components/BuilderToolbar/{BuilderToolbar.tsx, BuilderToolbar.styles.ts}` (publish toggle wired to mock state + a "Published"/"Draft" badge).

TEST — YOU

Toggle Publish in the Builder → badge flips; visit `/public/mock-id` → same files render read-only, no code editor, no chat panel visible.

CHUNK 3 — Backend Skeleton + Auth (Phases 7–8)

Phase 7 — FastAPI Skeleton + Motor + Atlas Connection

Deliverable: an empty but real, connected API with auto-docs.

MANUAL: NONE (ATLAS ALREADY SET UP IN PHASE 0)

CODE — US

Files: `server/app/{main,config,database,models}.py`, `server/requirements.txt`, `server/.env`.

```
mkdir server
cd server
py -3.13 -m venv venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

`.env` receives the five Phase-0 values plus `JWT_ALGORITHM=HS256`, `JWT_EXPIRE_MINUTES`, `OPENROUTER_BASE_URL=https://openrouter.ai/api/v1`, `CORS_ORIGIN=http://localhost:5173`, `COOKIE_NAME=webgenie_token`. `database.py` creates the Motor client and exposes `db`. `models.py` defines Pydantic mirrors of the `users` / `projects` documents.

TEST — YOU

Open `http://localhost:8000/docs` → Swagger UI loads. Hit a temporary health route that calls `db.list_collection_names()` → confirms the Atlas connection string actually works.

Phase 8 — Real Authentication APIs

Deliverable: real accounts, real sessions — mock mode retired.

MANUAL: NONE

CODE — US

Files: `server/app/auth/{security,dependencies}.py`, `server/app/routers/auth_router.py`, `server/app/main.py` (mount router); client: `src/store/slices/authSlice.ts` + `src/services/authService.ts` (swap mock thanks for real `api.post('/auth/ ...')` calls).

TEST — YOU

Register a real account via the UI → open Atlas → confirm a new document in `users` with a bcrypt hash, not plaintext. Refresh the page → still logged in (`/api/auth/me` works via cookie). Log out → cookie cleared, redirected to `/login`.

CHUNK 4 — AI Generation Pipeline (Phases 9–10, the hardest backend work)

Phase 9 — AI Site Generation (POST /api/projects + WebSocket)

Deliverable: a real prompt produces a real, working multi-file site, live.

MANUAL: NONE

CODE — US

Files: `server/app/ai/{schemas,prompts,service,normalizer,validator}.py`, `server/app/routers/{projects_router,ws_router}.py`; client: `vite.config.ts` (add the `/ws` proxy from §8), `src/services/{projectService,socketService}.ts`, `src/pages/BuilderPage/BuilderPage.tsx` (real WS, typed events, replacing Phase 5's mock data).

In `service.py`: plan phase is one `instructor` call against `FilePlanSchema` — if the model's plan omits `/App.js` or `/styles.css`, inject them manually rather than trusting the schema. Generation phase runs concurrently via `asyncio.Semaphore(6)` + `asyncio.gather`, each file normalized then validated; up to 2 retry rounds for any failing file; if `/App.js` still fails, insert a static placeholder component so the project is never totally unrenderable.

TEST — YOU

Type a real prompt on the Home page → watch `plan` then per-file `file_start` / `file_complete` events stream into the Builder UI live → ends with `done` → Sandpack renders the actual AI-generated site. Check Atlas: the project document shows `status: "completed"` with populated `files`.

Phase 10 — OpenRouter Model Tuning

Deliverable: reliable, good-looking generations across varied prompts. No new files.

MANUAL: NONE (UNLESS YOU WANT TO TRY A DIFFERENT FREE MODEL)

CODE — US

Run 5–10 generations back to back with varied prompts ("developer portfolio", "todo app", "bakery landing page"). Tune `extra_body={"provider": {"require_parameters": True}}`, the design-quality system prompt rules in `prompts.py` (light-mode-by-default, typography scale, single-file rule for small apps), and retry/concurrency settings if the free-tier rate limit gets hit.

TEST — YOU

Run the batch of test prompts yourself — each should finish `status: completed` with no placeholder-fallback triggered, and the generated sites should look reasonably clean by default.

CHUNK 5 — Revision Agent (Phase 11)

Phase 11 — Agent Chat API (revisions) + Manual Save

Deliverable: "make the header dark blue" actually only changes the header.

MANUAL: NONE

CODE — US

Files: `server/app/ai/diff.py` (`apply_operations`, md5 hashing), `schemas.py` (add `RevisionResultSchema` / `FileOpSchema`), `service.py` (add `revise_project`), `ws_router.py` (add chat WS), `projects_router.py` (add `PUT /files`); client: `src/components/ChatPanel/ChatPanel.tsx` (real WS, updates files via `projectSlice` + bumps local `version` to trigger the Sandpack remount from Phase 5), `src/hooks/useDebounce.ts` + `src/pages/BuilderPage/BuilderPage.tsx` (manual edits → debounce → `PUT /files`).

`revise_project` context sent to the AI: a file manifest (path + hash + size only, **not** full content, to save tokens) for every file, full content only for files judged relevant to the request, plus the last 3 chat messages. One `instructor` call against `RevisionResultSchema` returns ops; `diff.py` applies them with exact-match search/replace first, falling back to whitespace-normalized matching. Bumps `version`, appends both the user prompt and an assistant summary to `messages`, persists, streams result, closes socket.

TEST — YOU

- Type "change the button color to green" in ChatPanel → only the relevant file changes (check Atlas: untouched files keep the same `hash`).
- Manually edit code in the Sandpack editor, wait for the debounce, reload the page → your edit persisted.

CHUNK 6 — Integration & Dashboards (Phase 12)

Phase 12 — Listing, Community, Publish, Delete

Deliverable: the full product loop works end-to-end.

MANUAL: NONE

CODE — US

Files: `projects_router.py` (add list / full-get / publish / delete), `community_router.py` (public no-auth get); client: `src/services/projectService.ts` (real calls), `src/pages/{MyProjectsPage,CommunityPage,PublicPreviewPage}/Name.tsx` (real data), `components/BuilderToolbar/BuilderToolbar.tsx` (real publish call), delete with a confirm dialog.

TEST — YOU

The full loop, in order: register → prompt on Home → watch live generation → chat-revise it → manually edit a file → export the zip (confirm it runs standalone) → publish it → open `/public/{id}` in an incognito window with no login → confirm it renders read-only → delete the project → confirm it's gone from both Atlas and the Community list.

10. Final End-to-End Test Checklist

- ☐ Register + login + logout + session-persists-on-refresh all work
- ☐ Home page prompt → live streamed generation → completed site renders in Sandpack
- ☐ Manual code edit in Sandpack persists after reload
- ☐ Chat revision changes only the relevant file(s), preserves everything else's hash
- ☐ Export produces a zip that runs standalone with `npm install && npm run dev`
- ☐ Publish toggle works; public link opens read-only with no login required
- ☐ My Projects dashboard lists your projects; Community lists published ones from other/incognito sessions
- ☐ Delete removes the project from the database and both dashboards
- ☐ UI is visually consistent across every page (same fonts, colors, button/card styles from §7)

11. Resume Bullet (once built)

"Built a full-stack AI SaaS website builder (React/Vite + Sandpack + FastAPI + MongoDB) where users generate multi-file React websites from natural-language prompts, watch generation happen file-by-file in real time over WebSockets, revise the site through an AI chat agent that applies precise create/update/delete file operations, edit code manually in an embedded sandbox, and export or publish the result. Integrated an open-source LLM via OpenRouter's free tier; entire stack runs on free infrastructure (MongoDB Atlas, OpenRouter)."